# 数据结构与算法 A

## 2021 年期中 · 参考答案

本答案根据照片中可见讲解与独立计算整理。第二部分第 3、4 题缺失；第三部分第 1 题为结构性复原版本，其空号对应本次重排骨架。KMP 采用 0 起始下标，next[0]=-1，优化数组记 nextval。

## 一、选择填空

| 题号 | 答案 |
|---|---|
| 1 | C |
| 2 | B、D |
| 3 | A、C、D |
| 4 | $2^{n-1}$，C |
| 5 | 34 次 |
| 6 | D |
| 7 | C |
| 8 | 500、499、500、0 |

1．内层循环执行 $\lfloor \log_2 i \rfloor + 1$ 次，总计 $\Theta(n \log n)$。按教材的单位代价模型计数，忽略固定宽度整数溢出。

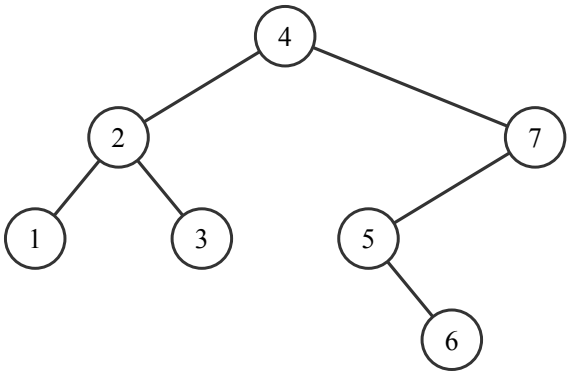
2．双向链表可沿 prev 访问；循环单链表可沿 next 绕一圈寻找前驱。若“循环链表”特指单向循环，结论仍成立。

3．同一对括号不会同时留在运算符栈中，故 C 不可能。入栈前应弹出当前优先级不低于待入运算符的栈顶运算符，故 A 与 D 不可能；括号可隔断优先级比较，B 可出现。

4．首个元素左右入队结果相同，其余 n-1 个元素各有左右两种选择，且得到的排列互异，故为 $2^{n-1}$。

5．元素数为 $(15 - 20 + 40) \bmod 40 = 35$，找最小值需 35-1=34 次比较。

6．由两遍历得到下图，结点 7 只有左子树、结点 5 只有右子树，不满足完全二叉树的层序连续性。




7．森林的左孩子—右兄弟表示中，原树没有孩子，等价于二叉树左指针为空。右指针反映兄弟关系，与是否为原树叶子无关。

8．完全二叉树中叶子数为 $\lceil \frac{1000}{2} \rceil = 500$。由 $n_0 = n_2 + 1$ 得 $n_2 = 499$；编号 1—500 有左孩子；只有右孩子的结点为 0 个。

# 二、辨析与简答

1．据原图字符表恢复：p=aabaac，t=aabaabaabaac。

普通 next：[-1,0,1,0,1,2]；优化 nextval：[-1,-1,1,-1,-1,2]。

```
下标  012345678901
文本  aabaabaabaac
模式  aabaac          i=5,j=5 处 b≠c，j←2
        aabaac         i=8,j=5 处 b≠c，j←2
          aabaac       全部匹配
起点  0 → 3 → 6
```

最终匹配起始下标为 6（从 1 开始编号为第 7 个字符）。失败时文本下标不回退。

2．不是最大堆：结点 6 的右孩子 8 大于 6。

自底向上检查 0 起始下标 4、3、2、1、0：

```
初始          23 17 14 6 13 10 1 5 8 12
处理下标4   23 17 14 6 13 10 1 5 8 12   无变化
处理下标3   23 17 14 8 13 10 1 5 6 12   交换6与8
处理下标2   23 17 14 8 13 10 1 5 6 12   无变化
处理下标1   23 17 14 8 13 10 1 5 6 12   无变化
处理下标0   23 17 14 8 13 10 1 5 6 12   无变化
```

最终树的层序为 {23,17,14,8,13,10,1,5,6,12}。

3、4．来源缺题，无法给出对应答案。

5．最终父指针数组为：

```
下标: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
父标: 0 0 0 0 0 0 0 6 0 0  0  9  0  0 12  0
```

全部元素属于以 0 为根的同一个集合，但父指针没有全部直接指向 0。路径压缩仅发生于实际执行 find 的路径；11→9→0、14→12→0、7→6→0 仍可能保留。若额外对所有元素 find 一遍，数组会全为 0，这不是题目要求的操作序列。

## 三、算法填空

1．对应重排版程序骨架：

```
// (1)
c >= '0' && c <= '9'
// (2)
stk[top]
// (3)
stk[top] = t[k]
// (4)
num = num * 10 + stk[digit_pos + 1] - '0'
```

遇到右括号，将最近左括号后的字符串逆序弹入临时数组，读出紧邻左括号之前的倍数，删除倍数，再按原顺序追加 num 次。原照片中的四条答案可辨为“数字判定”“栈顶不等于左括号”“将栈中字符写入临时串”“十进制累积数字”；因缺少中段，不能可靠映射到原卷各空。

复杂度说明：此复原版本逐层展开，若嵌套深度为 d、最终长度为 L，最坏可达 $O(dL)$；并非声称原题有线性时间要求。

2．按右—根—左的逆中序遍历：

```
// (5)
root->right != NULL && !root->right->visited
// (6)
root = root->right
// (7)
--k
// (8)
root->left != NULL && !root->left->visited
// (9)
root = root->left
```

判断中必须先检查指针非空，再访问 visited。每次首次访问结点才递减 k。此算法会修改 visited；重复查询前应重置标记。单次最坏时间 $O(n)$，遍历额外空间 $O(1)$（结点已有父指针与访问标记）。

## 四、算法设计与实现

1．双指针分别从两链表头出发，走到空指针时切换到另一链表头。两个指针走过的总长度相同，最终在首个公共结点或空指针相遇。

```
Node* firstCommon(Node* headA, Node* headB) {
    Node* p = headA;
    Node* q = headB;
    while (p != q) {
        p = p ? p->next : headB;
        q = q ? q->next : headA;
    }
    return p;
}
```

设两条独有前缀长度为 a、b，公共尾部长度为 c。切换后，长短前缀造成的差被抵消；无公共结点时，两指针最终同为空。时间 $O(a+b+c)$，额外空间 $O(1)$。

2．用 KMP 前缀函数 π。π[i] 表示 S[0..i] 的最长真前缀与后缀的公共长度。

先用 $O(n)$ 时间求 π。令 occ[1..n]=1，表示每个前缀在它自己的末尾出现一次。从长度 n 向 1 倒序，把长度 len 的出现次数加给其最长 border 长度 π[len-1]。所有 border 的包含关系形成失败树，倒序累加便能统计重叠出现次数。

```
pi = prefix_function(S)
occ[0..n] = 0
for len = 1..n:
    occ[len] = 1
for len = n down to 1:
    occ[pi[len-1]] += occ[len]
answer = max(len * occ[len] for len = 1..n)
```

正确性：每个结束位置对应的前缀，以及它沿失败链得到的全部 border，恰是该位置结束的所有前缀匹配。把计数沿失败树向父亲汇总，逐一覆盖这些匹配，不漏不重。

例 S=cdcdcdc，π=[0,0,1,2,3,4,5]，次数为 [4,3,3,2,2,1,1]，最大值为 $5 \times 2 = 10$。

时间 $O(n)$，额外空间 $O(n)$；乘积建议使用 64 位整数。